

Cover Group	Stats	Measurement	Evaluation
adder.A.a Channel A	100 %	Bins the input operand a_A ranging from 0–255 split into 11 bins. It ensures stimulus exercised the full dynamic range of a_A	100 % - Hit all 11 ranges , proving full distribution coverage of operand a_A
adder.A.b Channel A	100 %	Drives both: - sum_async_A = a_A + b_A - sum_sync_A = sum_sync_A + b_A (accumulator)	100% Means all bins for b_A were exercised. Full coverage ensures both async and sync paths see all ranges
adder.A.cross_a_b Channel A	100 %	cross coverage between a_A and b_A : 11 bins for a_A 11 bins for b_A 121 combinations Proves stimulus exercised the full 2-D space of operand pairs.	100% Means every combination of a_A and b_A ranges was hit. This is extremely strong coverage — it proves that random + constrained sequences are excellent.
adder.B.a Channel B	100 %	Bins the input operand B_a ranging from 0–255 split into 11 bins. It ensures stimulus exercised the full dynamic range of B_a	100 % - Hit all 11 ranges , proving full distribution coverage of operand B_a
adder.B.a Channel B	100 %	Drives both: - sum_async_A = a_A + b_A - sum_sync_A = sum_sync_B + b_A (accumulator) Same as Channel A, but for Channel B. Why it matters - It proves your dual-agent environment is stimulating both channels equally.	100% Means all bins for B_a were exercised. Full coverage ensures both async and sync paths see all ranges
adder.A.cross_a_b Channel B	100 %	cross coverage between b_A and b_A : 11 bins for a_A 11 bins for b_A 121 combinations Proves stimulus exercised the full 2-D space of operand pairs. Why it matters - It proves your dual-agent environment is stimulating both channels equally.	100% Means every combination of a_A and b_A ranges was hit. This is extremely strong coverage — it proves that random + constrained sequences are excellent.
Behavioral / Temporal / Validity Coverage These are the important ones — they measure DUT behavior, not just stimulus.			
adder.cross_overflow_valid	100 %	Cross between: overflow condition (sum_async > 255) valid_in == 1 Proves Testbench tested: - valid transactions that overflow - valid transactions that do not overflow - invalid transactions that overflow (illegal) - invalid transactions that do not overflow	4/4 bins hit (100.0%) All four combinations occurred and tested.
adder.cross_pattern_overflow	50 %	Cross between: - operand pattern class (4 bins: small, mid, large, edge) overflow flag Why it matters - It checks whether overflow occurs across different operand patterns.	4/8 bins hit (50.0%) Why only 50% Hit all operand patterns, but only half of the pattern + overflow combinations. This is normal unless you force overflow in all pattern classes.

Cover Group	Stats	Measurement	Evaluation
adder.cross_sum_valid	60 %	Measures Cross between: sum_async range (10 bins) valid_in (0 or 1) Why it matters Ensures: - valid transactions produce sums across full range - invalid transactions also produce sums across full range (legal but not checked)	60% means Covered most combinations but not all sum ranges under both valid states.
adder.cross_valid_rise_sum	60 %	Measures Cross between: valid_in rising edge sum_async range Why it matters This is a temporal coverpoint: - It checks the cycle where valid_in rises. - It ensures the sum at that moment spans the full range.	60% means Hit most sum ranges at the moment valid_in rises, but not all.
adder.valid_rst_illegal	50 %	Illegal condition: valid_in=1 during reset Checks robustness and provides illegal stimulus detection.	50% means You hit one illegal case, but not both combinations.
adder.overflow	100 %	Simple overflow coverpoint: - no overflow - overflow Ensure both behaviors occur.	100% means Hit both overflow and non-overflow cases.
adder.patterns	100 %	Measures Operand pattern classes: - small values - mid values - large values - edge values (0, 255) Ensures stimulus spans all operand categories.	100% means All pattern classes were exercised.
adder.rst	100 %	Evaluates all Reset states: - reset=0 - reset=1 Ensures monitor sees both reset and non-reset cycles.	100% means Reset behavior is covered.
adder.sum_sync	60 %	Checks Accumulator (sum_sync) range coverage. Accumulator grows slowly, so hitting all bins is harder.	60% means Hit most accumulator ranges but not all. May need longer tests or directed sequences to hit full accumulator range.

Summary Table

Coverage Group	What It Measures	Why It Matters	Interpretation
a, b	Operand range	Stimulus quality	Full range hit
cross_a_b	Operand combinations	2-D stimulus	Full matrix hit
overflow	Overflow behavior	Async sum correctness	Both cases hit
patterns	Operand classes	Stimulus diversity	All classes hit
sum_sync	Accumulator range	Sync path coverage	Partial
valid_in	Valid/invalid cycles	Control coverage	Full
valid_rise	Temporal valid edge	Driver correctness	Full
valid_rst_illegal	Illegal valid/reset	Robustness	Partial
cross_sum_valid	Sum vs valid	Behavioral coverage	Partial
cross_valid_rise_sum	Sum at valid rise	Temporal behavior	Partial
cross_pattern_overflow	Patterns vs overflow	Edge-case behavior	Partial